

Python Programming

Day 2: 반복문과 함수

`for, while, break, continue`, 함수, 반환값, 변수의 범위

Contents

1	학습 목표	3
2	반복문이 필요한 이유	3
3	for 반복문	3
3.1	들여쓰기와 코드 블록	4
4	range()와 정수 범위	4
4.1	range(stop)	5
4.2	range(start, stop)	5
4.3	range(start, stop, step)	5
4.4	왜 끝값을 포함하지 않는가	5
5	리스트와 문자열 순회	6
5.1	리스트 순회	6
5.2	문자열 순회	6
6	while 반복문	7
6.1	상태 변화와 무한 반복	7
6.2	for와 while의 선택	8
7	반복 제어: break	8
7.1	while True와 break	8
8	반복 제어: continue	9
8.1	홀수 건너뛰기	9
8.2	break와 continue 비교	9
9	함수의 필요성과 정의	10
9.1	함수 정의와 호출	10
9.2	매개변수와 인자	10
10	반환값: return	11
10.1	print()와 return의 차이	11
10.2	return 이후의 코드	12
11	변수의 범위: Scope	12
11.1	지역변수	12
11.2	전역변수	13
11.3	같은 이름의 지역변수와 전역변수	13
11.4	전역변수 수정과 global	13
12	종합 실습: practice_day02.py	14
12.1	1부터 100까지 출력	14
12.2	1부터 100까지의 합	14
12.3	구구단 7단	14
12.4	별 삼각형	14
12.5	3의 배수 건너뛰기	15
12.6	제곱 함수와 평균 함수	15
12.7	1부터 n까지의 합	15
12.8	함수를 이용한 덧셈 계산기	15
12.9	숫자 패턴 출력	15

12.10이름 선택 시 주의점	16
13 실습 파일 목록	16
14 핵심 요약	16
15 연습 문제	17

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 반복문이 필요한 이유
- for문과 range()의 동작 방식
- 리스트와 문자열을 순회하는 방법
- while문의 조건 기반 반복
- break와 continue의 차이
- 함수를 정의하고 호출하는 방법
- 매개변수와 인자의 역할
- print()와 return의 차이
- 지역변수와 전역변수의 범위
- 반복문과 함수를 결합한 간단한 프로그램 작성

2. 반복문이 필요한 이유

프로그램을 작성하다 보면 같은 작업을 여러 번 수행해야 하는 경우가 많다. 예를 들어 같은 문장을 다섯 번 출력하려면 반복문을 배우기 전에는 다음과 같이 작성할 수 있다.

```
1 print("Hello")
2 print("Hello")
3 print("Hello")
4 print("Hello")
5 print("Hello")
```

이 코드는 실행되지만 좋은 구조는 아니다. 반복 횟수가 100번, 1000번으로 늘어나면 코드가 지나치게 길어지고, 출력 문장을 수정할 때도 모든 줄을 직접 바꾸어야 한다.

반복문을 사용하면 같은 작업을 간결하게 표현할 수 있다.

```
1 for i in range(5):
2     print("Hello")
```

반복문은 단순히 코드를 줄이는 도구가 아니다. 여러 데이터에 같은 계산을 적용하거나, 조건이 만족될 때까지 작업을 계속하거나, 수치 계산을 누적하는 핵심 제어 구조이다.

핵심 포인트

반복문은 “같은 코드를 복사하는 문법”이 아니라, 반복되는 계산 과정을 하나의 규칙으로 표현하는 문법이다.

3. for 반복문

Python의 for문은 반복 가능한 객체의 원소를 하나씩 꺼내어 같은 코드 블록을 실행한다.

```
1 for variable in iterable:
2     statement
```

여기서 `iterable`은 원소를 차례대로 꺼낼 수 있는 객체를 의미한다. 대표적으로 `range`, 리스트, 문자열이 있다.

가장 기본적인 예제는 다음과 같다.

```
1 for i in range(5):
2     print(i)
```

출력 결과:

```
0
1
2
3
4
```

실행 과정은 다음과 같이 이해할 수 있다.

반복	i에 저장되는 값	실행되는 코드
1회	0	<code>print(0)</code>
2회	1	<code>print(1)</code>
3회	2	<code>print(2)</code>
4회	3	<code>print(3)</code>
5회	4	<code>print(4)</code>

3.1. 들여쓰기와 코드 블록

Python에서는 들여쓰기가 문법의 일부이다.

```
1 for i in range(3):
2     print(i)
3     print("inside loop")
4
5 print("outside loop")
```

들여쓰기된 두 줄은 매 반복마다 실행되지만, 마지막 줄은 반복문이 끝난 뒤 한 번만 실행된다.

핵심 포인트

Python에서는 중괄호 대신 들여쓰기로 코드 블록을 구분한다. 같은 블록에 속하는 코드는 동일한 깊이로 들여써야 한다.

실습

01_for.py에서 다음 출력이 나오도록 작성한다.

```
현재 숫자는 0입니다.
현재 숫자는 1입니다.
현재 숫자는 2입니다.
현재 숫자는 3입니다.
현재 숫자는 4입니다.
```

4. range()와 정수 범위

`range()`는 일정한 규칙으로 정수를 생성하는 객체이다. `for`문과 함께 매우 자주 사용한다.

4.1. range(stop)

```
1 for i in range(5):  
2     print(i)
```

range(5)는 0부터 5 직전까지의 정수, 즉 0, 1, 2, 3, 4를 차례대로 제공한다.

4.2. range(start, stop)

```
1 for i in range(3, 8):  
2     print(i)
```

출력:

```
3  
4  
5  
6  
7
```

첫 번째 값은 포함하고 두 번째 값은 포함하지 않는다.

4.3. range(start, stop, step)

```
1 for i in range(2, 11, 2):  
2     print(i)
```

출력:

```
2  
4  
6  
8  
10
```

세 번째 인자는 증가량이다. 음수를 사용하면 감소하는 범위를 만들 수 있다.

```
1 for i in range(10, 0, -1):  
2     print(i)
```

4.4. 왜 끝값을 포함하지 않는가

끝값을 포함하지 않는 규칙은 길이와 인덱스를 다룰 때 편리하다. range(5)가 정확히 다섯 개의 값 0부터 4까지 생성하므로, 리스트의 인덱스와 자연스럽게 맞는다.

$$\#\{0, 1, 2, 3, 4\} = 5$$

또한 일반적으로 양의 간격에서 생성되는 항의 개수는 구간 길이와 직접 연결된다.

체크 질문

range(1, 10, 3)이 만드는 값은 무엇인가? 마지막 값 10은 포함되는가?

실습

02_range.py에서 다음 두 코드를 작성한다.

1. 3부터 7까지 출력
2. 10부터 1까지 역순 출력

5. 리스트와 문자열 순회

Python의 for문은 숫자를 반복하는 문법이 아니다. 반복 가능한 객체의 원소를 하나씩 꺼내는 문법이다.

5.1. 리스트 순회

```
1 fruits = ["apple", "banana", "orange"]
2
3 for fruit in fruits:
4     print(fruit)
```

출력:

```
apple
banana
orange
```

첫 번째 반복에서는 fruit가 "apple"을, 두 번째 반복에서는 "banana"를, 세 번째 반복에서는 "orange"를 가리킨다.

반복 변수의 이름은 자유롭게 정할 수 있지만, 원소의 의미를 드러내는 이름을 사용하면 코드를 읽기 쉽다.

```
1 scores = [85, 90, 78]
2
3 for score in scores:
4     print(score)
```

5.2. 문자열 순회

문자열은 문자들의 순서 있는 모음이므로 한 글자씩 순회할 수 있다.

```
1 for character in "Physics":
2     print(character)
```

출력:

```
P
h
y
s
i
c
s
```

핵심 포인트

for x in data는 “data에서 원소를 하나 꺼내 x에 연결한 뒤 코드 블록을 실행한다”는 의미이다.

실습

03_for_list.py에서 다음 리스트의 원소를 문장 형태로 출력한다.

```
1 numbers = [3, 6, 9, 12, 15]
```

목표 출력은 현재 숫자는 3입니다.와 같은 형식이다.

6. while 반복문

while문은 조건이 참인 동안 코드 블록을 반복한다.

```
1 while condition:
2     statement
```

예제:

```
1 count = 1
2
3 while count <= 5:
4     print(count)
5     count += 1
```

출력:

```
1
2
3
4
5
```

실행 흐름은 다음과 같다.

1. 조건 count <= 5를 검사한다.
2. 조건이 참이면 코드 블록을 실행한다.
3. count += 1로 상태를 바꾼다.
4. 다시 조건을 검사한다.
5. 조건이 거짓이면 반복을 종료한다.

6.1. 상태 변화와 무한 반복

다음 코드는 종료되지 않는다.

```
1 count = 1
2
3 while count <= 5:
4     print(count)
```


count가 바뀌지 않으므로 조건은 계속 참이다. 이처럼 반복 조건이 영원히 참으로 유지되는 구조를 무한 루프라고 한다.

핵심 포인트

while문을 작성할 때는 “어떤 코드가 조건을 거짓으로 바꾸는가?”를 반드시 확인해야 한다.

6.2. for와 while의 선택

상황	일반적으로 적합한 반복문
반복 횟수를 알고 있음	for
리스트나 문자열의 원소를 처리함	for
반복 횟수를 미리 알 수 없음	while
특정 조건이 만족될 때까지 반복함	while

실습

04_while.py에서 while문을 이용하여 10부터 1까지 출력하고, 이어서 2부터 10까지 짝수만 출력한다.

7. 반복 제어: break

break는 현재 실행 중인 반복문을 즉시 종료한다.

```

1 for i in range(10):
2     if i == 5:
3         break
4
5     print(i)
```

출력:

```

0
1
2
3
4
```

i가 5가 되면 break가 먼저 실행되므로 print(i)는 실행되지 않는다.

7.1. while True와 break

조건을 반복문 안에서 검사하고 싶을 때 다음 구조를 자주 사용한다.

```

1 while True:
2     answer = input("Enter the answer: ")
3
4     if answer == "python":
5         break
```

while True 자체는 무한 반복이지만, 정답을 입력하면 break가 반복문을 종료한다.

핵심 포인트

break는 조건문을 종료하는 것이 아니라 자신을 감싸는 가장 가까운 반복문 하나를 종료한다.

실습

05_break.py에서 1부터 100까지 반복하되, 값이 50이 되는 순간 종료한다. 출력의 마지막 값이 49가 되는 이유도 설명한다.

8. 반복 제어: continue

continue는 반복문 전체를 종료하지 않는다. 현재 반복에서 남은 코드를 건너뛰고 다음 반복으로 이동한다.

```
1 for i in range(6):
2     if i == 3:
3         continue
4
5     print(i)
```

출력:

```
0
1
2
4
5
```

i == 3일 때 continue가 실행되어 아래의 print(i)를 건너뛴다. 그러나 반복문은 계속된다.

8.1. 홀수 건너뛰기

```
1 for i in range(1, 11):
2     if i % 2 == 1:
3         continue
4
5     print(i)
```

나머지 연산 i % 2가 1이면 홀수이다. 홀수인 경우만 건너뛰므로 짝수만 출력된다.

8.2. break와 continue 비교

명령어	현재 반복	이후 반복
break	즉시 중단	실행하지 않음
continue	남은 부분 건너뛰기	계속 실행

체크 질문

다음 코드에서 출력되는 값과 마지막 문장을 예측한다.

```
1 for i in range(5):
2     if i == 2:
3         continue
4     print(i)
5
6 print("Loop finished")
```

실습

06_continue.py에서 1부터 20까지 순회하며 3의 배수만 건너뛰다.

9. 함수의 필요성과 정의

함수는 특정 작업을 수행하는 코드를 하나의 이름 아래 묶은 것이다. 같은 작업을 여러 번 사용할 수 있고, 프로그램을 작은 단위로 나누어 구조화할 수 있다.

함수를 사용하지 않고 같은 코드를 반복하면 다음과 같다.

```
1 print("Hello")
2 print("Welcome")
3
4 print("Hello")
5 print("Welcome")
```

함수로 묶으면 다음과 같다.

```
1 def greeting():
2     print("Hello")
3     print("Welcome")
4
5
6 greeting()
7 greeting()
```

9.1. 함수 정의와 호출

함수의 기본 구조는 다음과 같다.

```
1 def function_name():
2     statement
```

def는 define의 약자이다. 함수를 정의하는 코드는 기능을 등록할 뿐 즉시 실행하지 않는다.

```
1 def hello():
2     print("Hello")
```

위 코드만 실행하면 출력이 없다. 실제로 함수를 실행하려면 호출해야 한다.

```
1 hello()
```

핵심 포인트

함수 정의는 “어떤 일을 할지 저장”하는 단계이고, 함수 호출은 “그 일을 지금 실행”하는 단계이다.

9.2. 매개변수와 인자

함수에 값을 전달하려면 매개변수를 사용한다.

```
1 def hello(name):
2     print("Hello", name)
3
4
5 hello("Kim")
6 hello("Lee")
```

함수를 정의할 때의 `name`을 매개변수(parameter), 호출할 때 전달하는 "Kim"을 인자(argument)라고 한다.

여러 매개변수를 사용할 수도 있다.

```
1 def add(a, b):
2     print(a + b)
3
4
5 add(3, 5)
```

실습

07_function.py에서 구분선을 출력하는 `line()` 함수를 만들고 다섯 번 호출한다.

10. 반환값: return

함수는 계산 결과를 호출한 곳으로 돌려줄 수 있다. 이때 `return`을 사용한다.

```
1 def add(a, b):
2     return a + b
3
4
5 result = add(3, 5)
6 print(result)
```

함수 호출 과정을 식처럼 생각할 수 있다.

`add(3, 5) → 8`

따라서 다음 대입은

```
1 result = add(3, 5)
```

개념적으로 다음과 같다.

```
1 result = 8
```

10.1. print()와 return의 차이

다음 함수를 보자.

```
1 def double(x):
2     print(x * 2)
3
4
5 result = double(7)
6 print(result)
```

출력:

```
14
None
```

함수 내부의 `print()`는 14를 화면에 보여 주지만, 값을 함수 밖으로 전달하지 않는다. 명시적인 `return`이 없는 함수는 `None`을 반환한다.

반면 다음 함수는 계산 결과를 반환한다.

```

1 def double(x):
2     return x * 2
3
4
5 result = double(7)
6 print(result)

```

출력:

14

기능	print()	return
화면에 표시	수행함	직접 수행하지 않음
함수 밖으로 값 전달	수행하지 않음	수행함
변수에 결과 저장	불가능	가능
후속 계산에 사용	불가능	가능

10.2. return 이후의 코드

return이 실행되면 함수는 즉시 종료된다.

```

1 def test():
2     return 10
3     print("This line is not executed")

```

마지막 print()는 실행되지 않는다.

실습

08_return.py에서 다음 함수를 작성한다.

1. 한 수의 제곱을 반환하는 square(x)
2. 세 수의 평균을 반환하는 average(a, b, c)

11. 변수의 범위: Scope

변수의 범위(scope)는 변수를 사용할 수 있는 코드 영역을 의미한다. 이번 장에서는 지역변수와 전역변수를 구분한다.

11.1. 지역변수

함수 내부에서 만들어진 변수는 기본적으로 그 함수 내부에서만 사용할 수 있다.

```

1 def test():
2     number = 100
3     print(number)
4
5
6 test()

```

함수 밖에서 같은 이름을 사용하면 오류가 발생한다.

```

1 def test():
2     number = 100
3
4

```

```

5 test()
6 print(number)

```

`number`는 함수 내부의 지역변수이므로 함수 밖에서는 정의되지 않은 이름이다.

11.2. 전역변수

함수 밖에서 정의된 변수는 전역변수이다.

```

1 name = "Python"
2
3
4 def show():
5     print(name)
6
7
8 show()

```

함수는 전역변수를 읽을 수 있다.

11.3. 같은 이름의 지역변수와 전역변수

```

1 x = 5
2
3
4 def test():
5     x = 10
6     print(x)
7
8
9 test()
10 print(x)

```

출력:

```

10
5

```

함수 내부의 `x = 10`은 전역변수 `x`를 수정한 것이 아니라 새로운 지역변수를 만든 것이다. 같은 이름의 지역변수가 전역변수를 가리는 현상을 shadowing이라고 한다.

11.4. 전역변수 수정과 `global`

전역변수를 함수 내부에서 수정하려면 `global` 선언이 필요하다.

```

1 count = 0
2
3
4 def increase():
5     global count
6     count += 1
7
8
9 increase()
10 print(count)

```

그러나 전역변수를 많이 수정하는 구조는 코드의 상태 변화를 추적하기 어렵게 만든다. 가능하면 반환값을 이용하는 편이 좋다.

```

1 def increase(count):
2     return count + 1
3
4
5 count = 0
6 count = increase(count)
7 print(count)

```

핵심 포인트

지역변수는 함수 내부의 독립적인 작업 공간을 만든다. 함수가 외부 상태를 무분별하게 수정하지 않도록 해 주기 때문에 프로그램의 구조를 명확하게 만든다.

실습

09_scope.py에서 지역변수와 전역변수가 같은 이름을 가질 때 각각 어떤 값이 출력되는지 확인한다.

12. 종합 실습: practice_day02.py

Day 2의 종합 실습은 반복문과 함수를 함께 사용하여 작성한다.

12.1. 1부터 100까지 출력

```

1 for number in range(1, 101):
2     print(number)

```

12.2. 1부터 100까지의 합

```

1 sum_1_to_100 = 0
2
3 for number in range(1, 101):
4     sum_1_to_100 += number
5
6 print(sum_1_to_100)

```

변수에 이전 결과를 계속 더하는 방식을 누적(accumulation)이라고 한다.

12.3. 구구단 7단

```

1 for number in range(1, 10):
2     print(f"7 x {number} = {7 * number}")

```

12.4. 별 삼각형

```

1 for number in range(1, 6):
2     print("*" * number)

```

문자열과 정수의 곱셈은 문자열을 반복한다.

12.5. 3의 배수 건너뛰기

```
1 for number in range(1, 21):
2     if number % 3 == 0:
3         continue
4
5     print(number)
```

12.6. 제곱 함수와 평균 함수

```
1 def square(number):
2     return number * number
3
4
5 def average(number1, number2, number3):
6     return (number1 + number2 + number3) / 3
```

12.7. 1부터 n까지의 합

```
1 def sum_to_n(n):
2     result = 0
3
4     for number in range(1, n + 1):
5         result += number
6
7     return result
```

12.8. 함수를 이용한 덧셈 계산기

```
1 def add(x, y):
2     return x + y
3
4
5 a = int(input("number 1: "))
6 b = int(input("number 2: "))
7
8 print("sum:", add(a, b))
```

12.9. 숫자 패턴 출력

```
1 for number in range(1, 6):
2     print(str(number) * number)
```

출력:

```
1
22
333
4444
55555
```


12.10. 이름 선택 시 주의점

Python에는 `sum()`이라는 내장 함수가 있다. 따라서 합계를 저장하는 변수 이름으로 `sum`을 사용하면 내장 함수를 가리게 된다.

```
1 sum = 0 # Not recommended
```

다음처럼 의미가 분명한 이름을 사용한다.

```
1 result = 0
2 sum_1_to_100 = 0
```

같은 파일에서 변수와 함수에 모두 `total`이라는 이름을 사용하는 것도 피하는 편이 좋다. 나중에 정의된 이름이 이전 객체를 덮어쓰기 때문이다.

13. 실습 파일 목록

실습

Day 2의 학습 파일은 다음과 같이 관리한다.

- 01_for.py
- 02_range.py
- 03_for_list.py
- 04_while.py
- 05_break.py
- 06_continue.py
- 07_function.py
- 08_return.py
- 09_scope.py
- practice_day02.py

14. 핵심 요약

1. `for`문은 반복 가능한 객체의 원소를 하나씩 꺼내어 처리한다.
2. `range(start, stop, step)`에서 `stop`은 포함되지 않는다.
3. 리스트와 문자열도 `for`문으로 순회할 수 있다.
4. `while`문은 조건이 참인 동안 반복한다.
5. `while`문에서는 조건을 변화시키는 코드가 없으면 무한 반복이 발생할 수 있다.
6. `break`는 반복문을 즉시 종료한다.
7. `continue`는 현재 반복의 남은 부분만 건너뛴다.
8. 함수는 `def`로 정의하고 함수 이름 뒤에 괄호를 붙여 호출한다.
9. 함수 정의는 실행이 아니며, 호출해야 함수 본문이 동작한다.

10. 매개변수는 함수가 입력을 받을 수 있게 한다.
11. `print()`는 값을 화면에 표시하고, `return`은 값을 호출한 곳으로 전달한다.
12. 반환값이 없는 함수는 `None`을 반환한다.
13. 지역변수는 함수 내부에서만 사용 가능하며, 전역변수와 같은 이름을 가질 수 있다.
14. 전역변수 수정에는 `global`을 사용할 수 있지만, 반환값을 이용한 구조가 일반적으로 더 명확하다.

15. 연습 문제

1. `range(2, 15, 4)`가 만드는 값을 모두 쓰고, 끝값 15가 포함되지 않는 이유를 설명하라.
2. `for`문과 `while`문을 선택하는 기준을 각각 예제와 함께 설명하라.
3. 1부터 50까지의 수 중 5의 배수만 출력하는 코드를 작성하라.
4. 1부터 30까지 반복하되 4의 배수는 건너뛰고, 25를 만나면 반복을 종료하는 코드를 작성하라.
5. 문자열 "Quantum"의 문자를 한 줄에 하나씩 출력하라.
6. 두 수 중 더 큰 값을 반환하는 `maximum(a, b)` 함수를 작성하라.
7. 양의 정수 `n`을 받아 1부터 `n`까지의 짝수 합을 반환하는 함수를 작성하라.
8. `print()`와 `return`의 차이를 변수 저장 가능 여부와 후속 계산 가능 여부를 중심으로 설명하라.
9. 지역변수와 전역변수가 같은 이름을 가질 때 Python이 어떤 변수를 사용하는지 설명하라.
10. `global`을 사용하지 않고 함수 호출 횟수를 증가시키는 구조를 반환값으로 작성하라.